Systemadministration

Linux

Bash Scripting

Agenda

1. Scripting
2. Hur skriver man ett bash-skript?
3. Varför “set -eou pipefail”?
4. Kort repetition: $PATH
5. Vart borde man placera sina skript?
6. Grundkoncept för skripting
7. Ett lite mer avancerat skript
8. Exit Codes
9. Vilkorliga satser
10. Vilkorliga satser med test
11. Särskilt användbara kommandon i skirpt
12. Tips

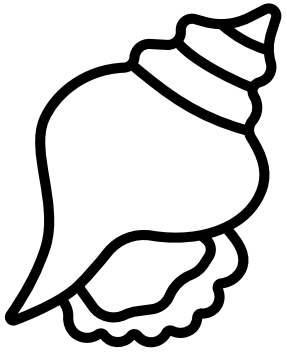
Scripting

GNU Bash

Vanligaste **skalet** (**shell**) för Linux

```
nics@linux:~$ command --flags arguments
```

Även ett skriptspråk



Skriptspråk

Ett programmeringsspråk som mestadels används för att automatisera annars manuella arbetsuppgifter

Program

Instruktioner till en dator

Definieras i ett programmeringsspråk

Programmering

Två typer av programmeringsspråk

- **Skriptspråk**
- **Systemspråk**

Skript

- Enklare syntax
- Kompileras inte i förväg
- Närmare användarmiljön

System

- Högre prestanda
- Kan lösa generella problem
- Kräver mer kod för motsvarande resultat

Skript	System
Bash	C
PowerShell	C++
Awk	Java
Batch (cmd)	Go
Expect	Rust
Perl	Perl
Python	Python

Hur skriver man ett Bash-skript?

1. Skapa en textfil*

Konventionellt används filändelsen ”**.sh**”

```
nics@linux:~$ touch script.sh
```

*och placera den på en lämplig plats

2. Gör filen exekverbar

```
nics@linux:~$ chmod 755 script.sh
nics@linux:~$ ls -l script.sh
-rwxr-xr-x  nics  nics  script.sh
```

3. Skriv dina instruktioner

```
nics@linux:~$ nano script.sh
```

Inleds med

```
#!/usr/bin/env bash
set -eou pipefail

# print "Hello World!"
echo "Hello World!"

# open new-file with nano
nano new-file
```

Livedemo

Hur man skriver ett enkelt skript

Med & utan ’shebang’

5. Kör ditt skript

4 sätt att köra skript på*:

```
nics@linux:~$ bash /home/nics/script.sh
nics@linux:~$ bash script.sh
nics@linux:~$ /home/nics/script.sh
nics@linux:~$ ./script.sh
```

Obs! "./"

*om man placerade skriptet på en olämplig plats, det finns ett enklare sätt

Varför ”set -eou pipefail"?

Interaktiv användning

Bash antar att du använder det interaktivt

```
viktor@debian:~$ ls nothing
ls: cannot access 'nothing': No such file or directory
viktor@debian:~$ ls
viktor@debian:~$
viktor@debian:~$
```

Om ett fel sker kan man fortsätta köra nya kommandon

Skriptad användning

Med ”**set -eou pipefail**”:

```
viktor@debian:~$ set -eou pipefail
viktor@debian:~$ ls nothing
ls: cannot access 'nothing': No such file or directory


Session stopped
    - Press <Return> to exit tab
    - Press R to restart session
    - Press S to save terminal output to file
```

Så avslutas **Bash** vid fel

Skriptad användning

För att enklare upptäcka fel bör man alltid inleda med:

```
#!/usr/bin/env bash

set -eou pipefail

# resterande skript:
```

Kort repetiton: $PATH

Environment **Var**iables

Inställningar hanteras i:

- **/etc/profile**
- **~/.profile**

Definierar bland annat:

- **$PATH**
- **$EDITOR**
- **$PAGER**

$PATH

Ex. Kommandot **ls** finns i **/usr/bin/**

Båda kör kommandot **ls**:

```
nics@linux:~$ /usr/bin/ls
nics@linux:~$ ls
```

Varför behöver jag inte skriva hela sökvägen?

```
nics@linux:~$ echo "$PATH"
/usr/local/bin:/usr/bin:/bin
```

$PATH och skript

Ex. Kommandot **ls** finns i **/usr/bin/**

```
nics@linux:~$ echo "$PATH"
/usr/local/bin:/usr/bin:/bin
nics@linux:~$ which ls
/usr/bin/ls
nics@linux:~$ ls
super-script.sh
nics@linux:~$ super-script.sh
super-script.sh: command not found
```

Vart borde man placera sina skript?

$PATH och skript

Det är mycket enklare om skriptet ligger i din **$PATH**

Då fungerar:

```
nics@linux:~$ script.sh
```

Vart ska man placera sina skript?

För bara din användare:

```
nics@linux:~$ nano ~/bin/script.sh
```

För alla användare:

```
nics@linux:~$ sudoedit /usr/local/bin/script.sh
```

För superusers (root/**sudo**):

```
nics@linux:~$ sudoedit /usr/local/sbin/script.sh
```

~/bin/?

Kommer inte förberett i alla distributioner

1. Skapa katalogen
2. Logga ut och sedan in igen

~/bin/ ska sedan finns i din **$PATH**

Rättigheter

Tilltänkta användare behöver ha **execute** till ditt skript

Men <u>aldrig</u> **write**!

Lämpliga rättigheter:

Location	Permissions & Ownership	Purpose
~/bin/	-rwxr-xr-x $USER $USER	För en specifik användare
/usr/local/bin/	-rwxr-xr-x root root	För flera användare
/usr/local/sbin/	-rwxr-xr-x root root	För superusers (root/sudo)

Rättigheter

*Varför ska man inte ge andra **write** till ett skript?*

Grundkoncept för skripting

Kommandon

Vanliga kommandon skrivs som vanligt

```
ls
pwd
```

Kommentarer

```
# kommentarer inleds med "#"
# har ingen effekt när skriptet körs
```

Strängar

Text

Kan t.ex. visas med kommandot **echo**

```
echo "123 detta är en sträng!"
echo "En sträng...
...kan vara mer än en rad"
```

Variabler

Något som kan ändra värde

Konventionellt uppercase

```
# en variabel med en sträng som värde
VAR="string"
```

Obs! inga mellanrum

Hur använder man variabler?

Variabler gör ett skript mer flexibelt

För att använda värdet: **"$VAR"** Obs! ""

```
# en variabel med en sträng som värde
VAR="string"

# använd värdet av variablen med: "$VAR"
echo "Variabeln har värdet: $VAR"

VAR="new value!"
```

Command Substitution

Kör ett kommando och använd dess output istället för att skriva det till konsolen

```
"$(whoami)"
```

Ex.

```
echo "Du är inloggad som: $(whoami)"
```

Command Substitution

Gör det inte svårare för dig själv

Båda gör samma sak:

```
# visa filer med "ls"
ls

# visa filer med "ls"
echo "$(ls)"
```

Ett lite mer avancerat skript

Livedemo

hwinfo.sh

```
#!/usr/bin/env bash
set -eou pipefail

HOSTNAME="$(hostname)"
MEMORY="$(free -h)"
UPTIME="$(uptime -p)"

echo "Host $HOSTNAME has been $UPTIME
 * Memory usage:
$MEMORY"
```

Exit Codes

Exit Codes

Visar om ett kommando lyckas efter det körts

0	Success ✓
≥1	Failure ✗

Variabeln **$?** innehåller senaste **exit code**

```
nics@linux:~$ echo "$?"
```

Exit Codes

Ex.

```
nics@linux:~$ ls    ✓
super-script.sh
nics@linux:~$ echo "$?"
0
nics@linux:~$
```

0	Success ✓
≥1	Failure ✗

Exit Codes

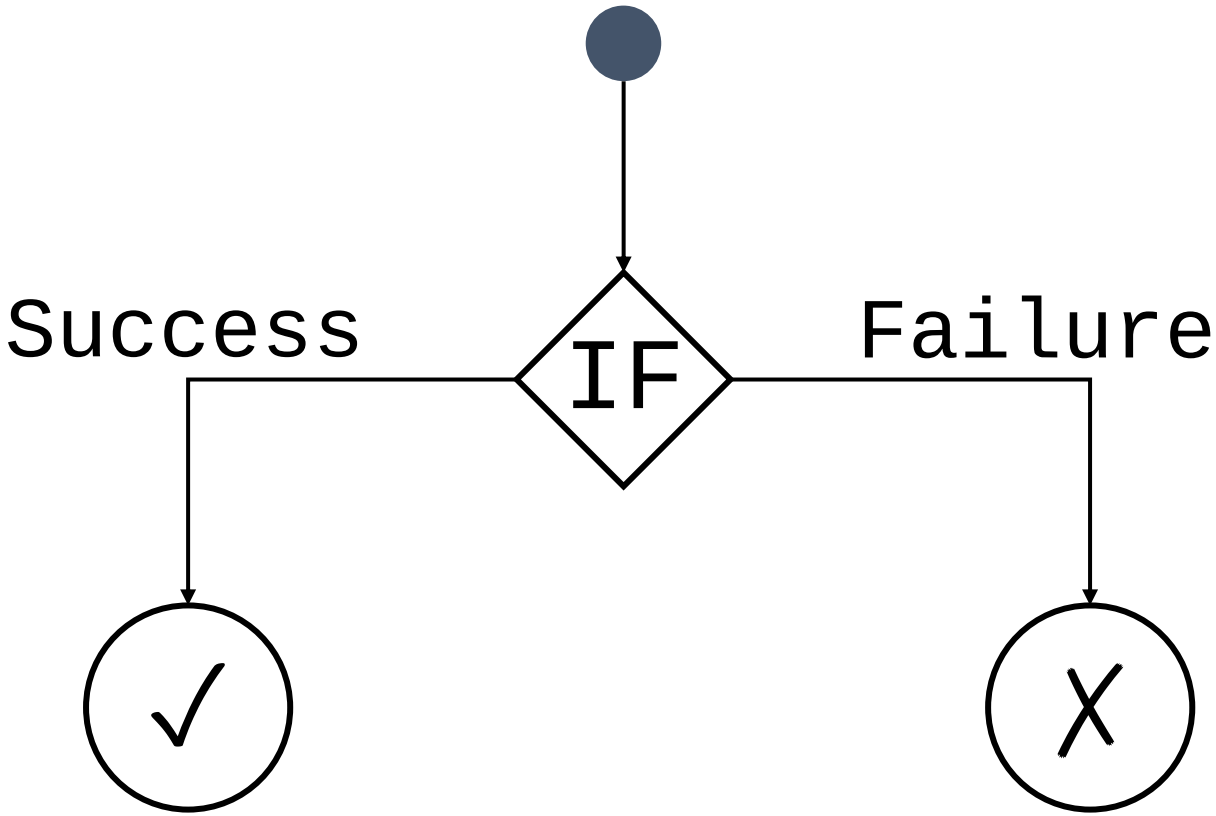
Ex.

```
nics@linux:~$ ls amazing-script.sh  ✗
ls: cannot access 'amazing-script.sh':
No such file or directory
nics@linux:~$ echo "$?"
2
nics@linux:~$
```

0	Success ✓
≥1	Failure ✗

Villkorliga satser

Conditional



if

Kör ett kommando, om det ger exit code 0 (✓) kör då följande kommandon

```
if ________
then
    echo "success!"
fi
```

```
if ________; then
    echo "success!"
fi
```

else

Om **if** inte gav exit code 0 (✓) kör då följande kommandon

```
if _______; then
    echo "success!"
else
    echo "failure!"
fi
```

elif

Om **if** inte gav exit code 0 (✓), kontrollera då om ett annat kommando ger exit code 0 (✓)

```
if _______; then
    echo "first check succeeded!"
elif _______; then
    echo "first check failed!"
    echo "second check succeeded!"
fi
```

Negated if/elif

Kör ett kommando, om det ger exit code ≥1

(X) kör då följande kommandon

```
if ! _______
then
    echo "failure!"
fi
```

```
if ! _______; then
    echo "failure!"
fi
```

Negated if

Ex.

```
# om gruppen wheel inte finns
if ! getent group wheel > /dev/null 2>&1; then
    # skapa gruppen wheel
    groupadd -r wheel
fi
```

Villkorliga satser med test

if/elif med filer/kataloger

Ex.

```
if [ -f "/etc/ssh/sshd_config" ]; then
    echo "Found SSH server config!"
fi
```

-a	✓ if file/directory exists
-d	✓ if directory exists
-f	✓ if file exists

if/elif ! med filer/kataloger

Kan även kombineras med ”!” för att kontrollera motsatsen

Ex.

```
if [ ! -f "/etc/ssh/sshd_config" ]; then
    echo "Did not find SSH server config!"
fi
```

if/elif med filrättigheter

Ex.

```
if [ ! -r "/var/log/messages" ]; then
    echo "Cannot access logs :("
fi
```

-r	✓ if you have read permission
-w	✓ if you have write permission
-x	✓ if you have execute permission

if/elif med strängar

Ex.

```
read -rp "Username: " USERNAME

# kontrollera om USERNAME är tomt
if [ -n "$USERNAME" ]; then
    echo "Hello $USERNAME"
fi
```

-n	✓ if string is not empty
-z	✓ if string is empty

Särskilt användbara kommandon i skript

type

Kontrollera om ett kommando finns

```
nics@linux:~$ type sudo
sudo is /usr/bin/sudo
nics@linux:~$ echo "$?"
0
```

```
nics@linux:~$ type doas
-bash: type: doas: not found
nics@linux:~$ echo "$?"
1
```

```
if ! type sudo > /dev/null 2>&1; then
    echo "Could not find sudo, installing..."
    apt install -y sudo
fi
```

grep

Sök efter mönster

```
nics@linux:~$ grep 172.23.121.21 /etc/network/interfaces
        address 172.23.121.21/24
nics@linux:~$ echo "$?"
0
nics@linux:~$ grep 192.0.2.5 /etc/network/interfaces
nics@linux:~$ echo "$?"
1
```

Tips: om du enbart behöver exit code kan du dölja output med ”**grep -q**”

exit

Avsluta ett skript innan sista instruktionen

```
if [ ! -f "/etc/ssh/sshd_config" ]; then
    echo "Did not find SSH server config!"
    echo "Unable to continue. Exiting!"
    exit 1
fi
```

Exit code

Tips

sudo

Undvik att använda **sudo** i dina skript

Det är ofta bättre om användaren kör hela skriptet med **sudo**

Särskilt viktigt för icke-interaktiva skript, dvs. vid schemaläggning

shellcheck

Kontrollerar syntax för **Bash**-skript

https://www.shellcheck.net/

```
viktor@debian:~$ shellcheck test.sh

In test.sh line 7:
if [ ! -f "/etc/ssh/sshd_config" ]
^-- SC1049 (error): Did you forget the 'then' for this 'if'?
^-- SC1073 (error): Couldn't parse this if expression. Fix to allow more checks.


In test.sh line 9:
fi
^-- SC1050 (error): Expected 'then'.
  ^-- SC1072 (error): Unexpected keyword/token. Fix any mentioned problems and try again.

For more information:
  https://www.shellcheck.net/wiki/SC1049 -- Did you forget the 'then' for thi...
  https://www.shellcheck.net/wiki/SC1050 -- Expected 'then'.
  https://www.shellcheck.net/wiki/SC1072 -- Unexpected keyword/token. Fix any...
viktor@debian:~$
```

Hur gör man?!

Du använder redan **Bash**!

Skriv dina kommandon till en fil → skript

Exempel tillvägagångssätt

1. Ta en ny checkpoint
2. Gör det manuellt
3. Skriv ner alla kommandon,
   steg för steg, till en skriptfil
4. Återgå till tidigare checkpoint
5. Provkör din skriptfil
   - Rätta till eventuella fel
   - Ta bort manuella prompter

3. Skriv ner alla kommandon

Hur?

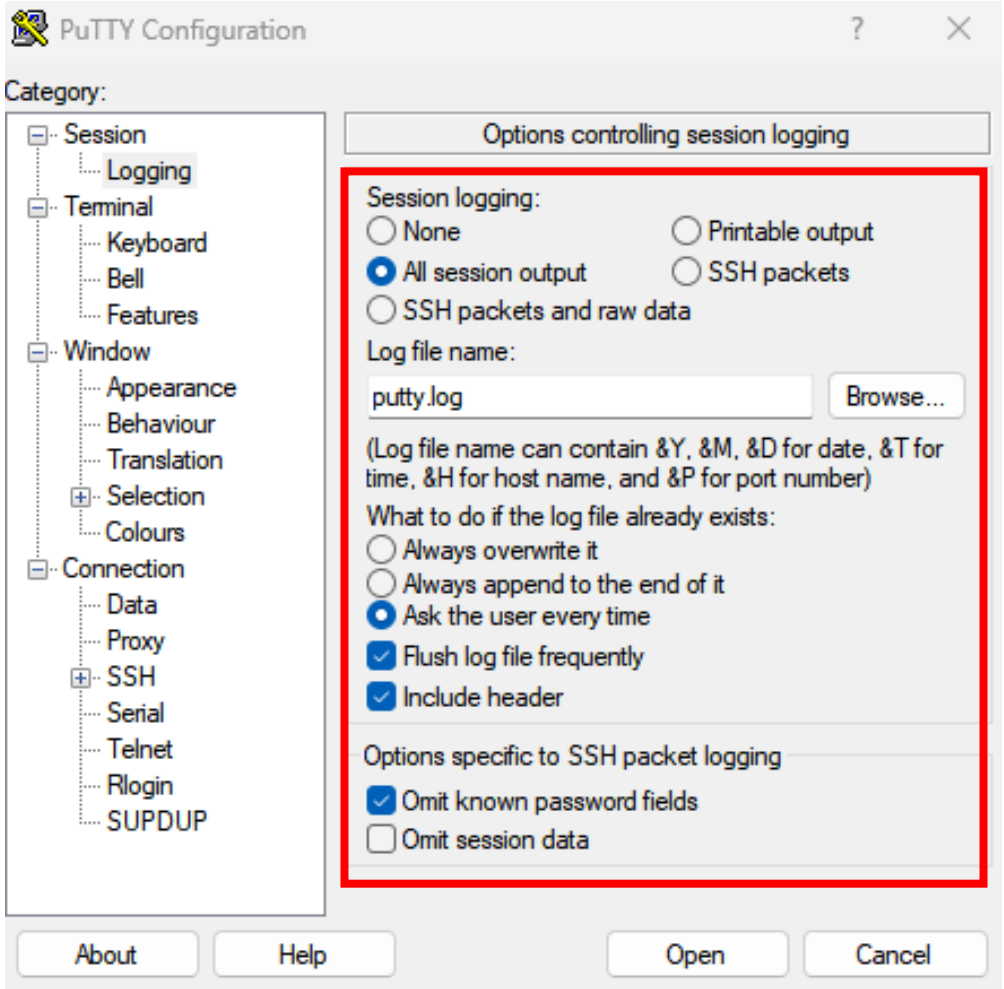
Alternativ 1:

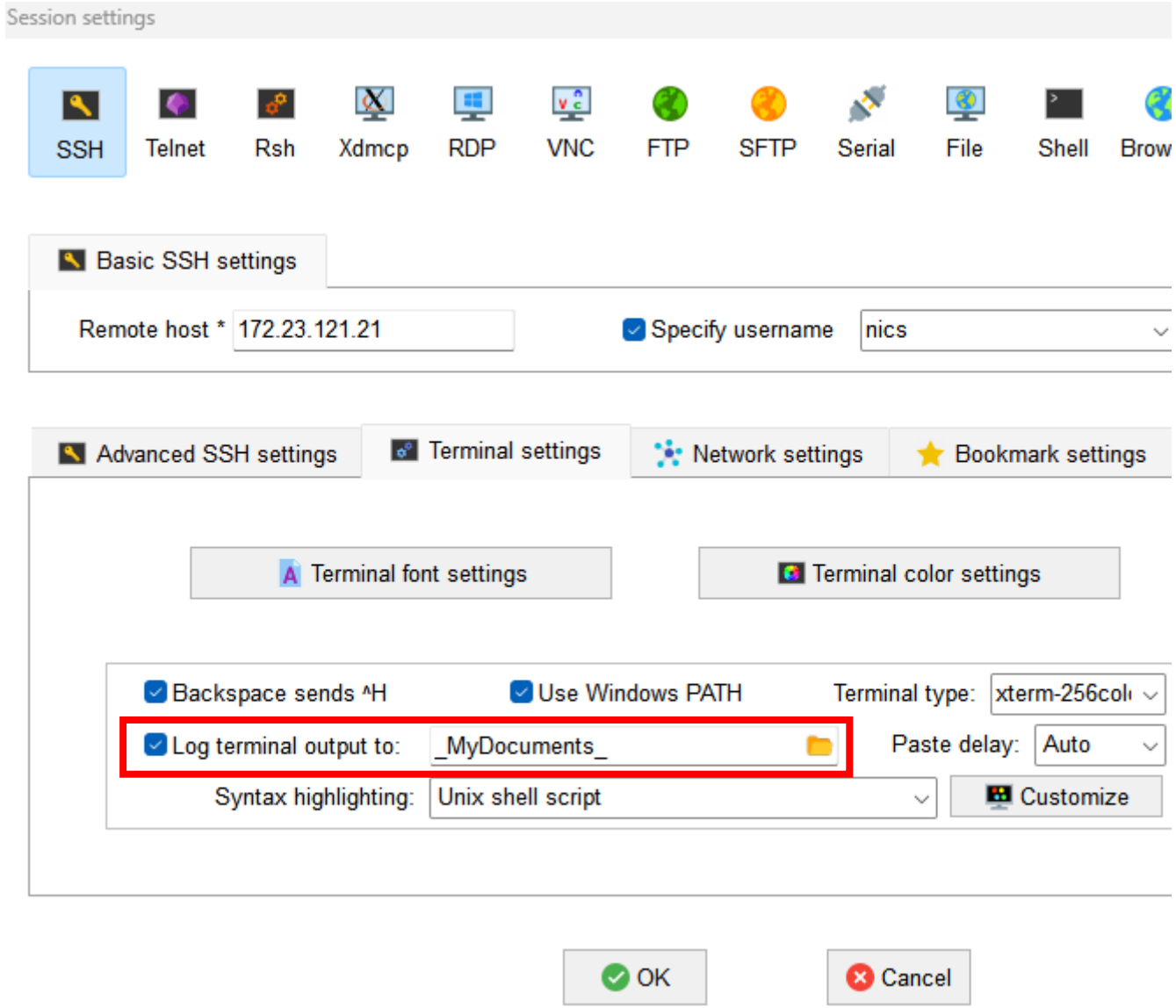
```
nics@linux:~$ history
nics@linux:~$ less ~/.bash_history
```

Alternativ 2:

Spela in i PuTTY/MobaXterm

Spela in med PuTTY/MobaXterm





Mitt skript fungerar inte!

…och jag vet inte varför

Klistra in för att se exakt vad ditt skript gör

```
#!/usr/bin/env bash

set -eou pipefail
trap 'read -p "[$LINENO] $BASH_COMMAND "' DEBUG
set -x
# resterande skript:
```

DEBUG

Kör skriptet rad för rad, och visar:

- Nuvarande radnummer
- Innehållet på raden
- Vilka kommandon som körs när raden exekveras
- Innehållet i variabler när de skapas/uppdateras

DEBUG

En nackdel: det visar även debug-’magin’ innan varje rad

```
nics@debian:~$ test.sh
[7] set -x
++ read -p '[9] HOSTNAME="$(hostname)" '
[9] HOSTNAME="$(hostname)"
++ hostname
+ HOSTNAME=debian
++ read -p '[10] UPTIME="$(uptime -p)" '
[10] UPTIME="$(uptime -p)"
++ uptime -p
+ UPTIME='up 1 day, 6 hours, 4 minutes'
++ read -p '[11] MEMORY="$(free -h)" '
[11] MEMORY="$(free -h)"
++ free -h
+ MEMORY='              total        used        free      shared  buff/cache   available
Mem:           365Mi       250Mi        64Mi       3.4Mi       193Mi       115Mi
Swap:          975Mi       268Ki       975Mi'
```

Till nästa gång

- Läs kurslitteraturen!
- Gör klart labb6!

Tack för idag